

Validatie-run protocol

stairlift live 3D telemetry

Nout Mulder • DeVi Comfort

Run-info (invullen vóór je begint):

Datum: _____ Tijd start: _____ Tijd eind: _____

Locatie / klant: _____

Lift serial / ID: _____

Software-versie: _____

Tester: _____

Begeleider aanwezig? ☐ ja ☐ nee

Overzicht

Eén rijbeweging op- en neer, één rail, vier metingen + twee observaties. Doel: elk DV1–DV3 criterium in de validatie-tabel van het verslag van een echt cijfer voorzien in plaats van [meet].

Belangrijk: dit is gewoon de lift normaal omhoog en omlaag rijden met de joystick/knoppen op de stoel. Geen firmware-calibratie mode (die CalibrationRunService flow in de code is voor installatie, niet voor validatie). De firmware kent zijn rail al; wij kijken alleen mee.

Wat we WEL testen

- Of onze `IpcEncoderMapper` cumulatief de juiste positie oplevert door pulse-counts van straight + bend segmenten te integreren
- Of de Catmull-Rom interpolatie en quaternion-compositie van de chair op het scherm overeenkomen met de werkelijkheid
- Of het Kalman-filter encoder-ruis significant reduceert
- Of de Bluetooth round-trip binnen het 100 ms budget blijft
- Of de hele pipeline live, end-to-end, soepel werkt

Wat we NIET testen

- Encoder-hardware (al door DeVi geverifieerd)
- Firmware pulse-cost constanten (al geverifieerd)
- Lengte van bochten met meetlint (kan niet accuraat; lengte wordt door de firmware berekend uit $\text{pulse-count} \times \text{bend-radius} \times \text{segment-hoek}$)

Dus: markers plak je **alleen op rechte stukken en bij de stations**. Bochten meten we niet — de software berekent ze, en indirect testen we of die berekening klopt door te kijken of de chair op de *volgende* marker (op een rechte na de bocht) komt waar verwacht.

Pass/fail logica:

DV1(b) pass: $T1 \leq 100 \text{ ms}$

DV2(b) pass: $\max(T4) \leq 5 \text{ mm op rail} \leq 5 \text{ m}; \leq 10 \text{ mm op langere rail}$

DV3(b) pass: $T2 / T3 \geq 10$ (factor tien jitter-reductie)

ID	Meting / observatie	Voor
T1	Bluetooth round-trip tijd (run-gemiddelde, ms)	DV1(b)
T2	Raw encoder jitter, stationair, 10 s (pulses)	DV3(b) baseline
T3	Kalman-filtered jitter, stationair, 10 s	DV3(b)
T4	Positie-error bij referentiepunten (mm)	DV2(b)
O1	Visuele smoothness (geen jumps, oscillation, snapping)	observation
O2	Anomalieën (BT drops, checksum errors, glitches)	observation

1 Materiaal-checklist

- ☐ Toegang tot één werkende fysieke stairlift
- ☐ Meetlint (rolmaat) met mm-graduatie, minimaal 5 m lang
- ☐ Schilderstape of stickertjes voor 4–5 markers
- ☐ Tablet met de app (UP-Studio debug build, recent geïnstalleerd)
- ☐ Tablet-oplader (alleen voor opladen achteraf)
- ☐ Telefoon voor foto's van het scherm bij elke marker (mag dezelfde zijn als de tablet als die alleen wordt)
- ☐ Pen + dit document uitgeprint
- ☐ Tijd: ~1 uur op de lift, daarna ~15 min thuis voor data-pull
- ☐ Voor de data-pull achteraf: laptop + USB-kabel + adb tools. Niet nodig tijdens de rijbeweging zelf.

2 Logfile-capture

De TestLogger autoload in de app schrijft alle validatie-events automatisch naar een file op de tablet, in de app-private external storage map (geen permissies nodig). Geen handmatig print-redirect of `adb logcat` vereist tijdens de rijbeweging zelf. Je hoeft de tablet niet met USB-kabel aan een laptop te koppelen tijdens het rijden — alleen *na* de run om het bestand te pullen.

2.1 Wat er automatisch gelogd wordt

Zodra je de app opent:

- [CSV] per encoder-meting (~20 Hz tijdens beweging)
- [BT_RTT] per request/response paar (round-trip in ms)
- [BASELINE_START] / [BASELINE_END] bij knop-druk
- [MARKER] per MARK-druk, inclusief encoder en positie
- [VALIDATION_SUMMARY] bij Print summary

Bestand op de tablet:

```
/storage/emulated/0/Android/data/com.devistairlifts/files/
validation_logs/session_<datetime>.log
```

In de app onderaan de *Validatie / Tests* sectie zie je de huidige bestandsnaam en het aantal regels.

2.2 Knop *Nieuw logfile*

Sluit het huidige bestand en opent een nieuw bestand. Counters van de sessie worden ook gereset. Gebruik dit als je een nieuwe sessie wilt starten (bv. na een testrun in demo-modus en vóór de echte run op de lift).

2.3 Na de rijbeweging: log pullen via adb

Tablet via USB aansluiten op de laptop, dan:

```
adb devices # bevestig dat tablet zichtbaar is
mkdir -p ~/logs/tablet
adb pull /storage/emulated/0/Android/data/com.devistairlifts/files/\
validation_logs/ ~/logs/tablet/
```

Alle session_*.log bestanden staan dan in ~/logs/tablet/.

2.4 Sanity-check vóór vertrek

- ☐ App opent zonder errors in Live-tab
- ☐ Onderaan Validatie-sectie zie je: *Log: session_*.log (N regels)*
- ☐ Connect via *Demo* entry → CSV samples teller in app loopt
- ☐ Klik **MARK** → markers teller loopt op, log-regels teller loopt op
- ☐ Klik **Start baseline** → status wordt oranje *BASELINE ACTIEF*
- ☐ Klik **Nieuw logfile** → bestandsnaam in label verandert

Sanity-check voltooid op (datum/tijd): _____

3 Op locatie — marker setup

3.1 Stap 0a — Lift uit en rail meten

- ☐ Lift uitgeschakeld, sleutel uit, hoofdschakelaar uit
- ☐ Meet de *totale* rail-lengte met meetlint (van bottom-stootblok tot top-stootblok)
- ☐ Tel het aantal horizontale bends visueel
- ☐ Schat globaal welke segment-typen (straight, up-bend, down-bend, h-left, h-right)

Totale rail-lengte: _____ mm

Aantal horizontale bends: _____

Aantal verticale bends: _____

Omschrijving rail (bv. “rechte traploep met één bocht onderaan”):

3.2 Stap 0b — Marker-posities kiezen en meten

Markers alleen op rechte stukken en bij de stations. Bochten sla je over — die kun je niet accuraat met meetlint meten.

Plan voor markers:

1. MP1 — vlak bij bottom station (op de eerste rechte, ~100–300 mm vanaf bottom-stootblok)
2. MP2 — midden van een lang recht stuk
3. MP3 — net na een bocht, op een recht stuk (zo test je of de software de bocht correct integreert)
4. MP4 — volgend recht stuk
5. MP5 — vlak bij top station (op de laatste rechte)

Per marker: plak een stukje tape op een recht railstuk. **Meet de afstand vanaf het begin van dat rechte stuk** (dus vanaf waar de vorige bocht eindigt, of vanaf het bottom-stootblok voor MP1). Dat is een rechte meting met de meetlint, dus accuraat.

Tip: zet markers op de buitenkant van de rail waar een herkenbaar punt op de chair (bv. wielkje, plaatje) duidelijk zichtbaar overheen rolt. Zo zie je tijdens de rijbeweging precies wanneer de marker gepasseerd wordt.

ID	Welk recht stuk?	Offset (mm)	Vanaf...
MP1			bottom-stootblok
MP2			einde vorige bocht
MP3			einde vorige bocht
MP4			einde vorige bocht
MP5			einde vorige bocht

Hoe je de absolute positie achteraf berekent: de verwachte positie van een marker = cumulatieve lengte van alle voorgaande rail-onderdelen (straights + bochten) + de gemeten offset. De firmware kent alle onderdeel-lengtes: een straight is direct in mm; een bocht is $r \cdot \Delta\alpha \cdot n_{segs} = 150 \cdot \frac{5\pi}{180} \cdot n \approx 13.09 \cdot n$ mm.

4 De rijbeweging zelf

4.1 Stap 0c — Opstart en verbinding (~10 min)

Wat je doet: tablet aan, app openen, verbinden met lift, nieuw logfile starten.

- ☐ Lift aan (sleutel om)
- ☐ Tablet aan, app openen (UP-Studio)
- ☐ Ga naar *Live*-tab (laatste knop in zijbalk)
- ☐ Bluetooth: kies de juiste lift uit de dropdown → tik **Connect**
- ☐ Wacht tot status *Verbonden* verschijnt (groen), lift-naam in oranje
- ☐ Tik **Nieuw logfile** in Validatie-sectie → start frisse sessie
- ☐ Onder Validatie-sectie zie je: *Log: session_YYYYMMDD_HHMMSS.log*
- ☐ *CSV samples* teller loopt op zodra encoder-data binnenkomt

Geen klok-tijden noteren — elke knop-druk schrijft zelf een millisecond-timestamp in het log.

Probleem-oplossing: Geen oplopende CSV samples teller? Controleer of status *Verbonden* is (groene tekst). App crash? Stop, herstart, noteer hieronder.

Eventuele opstart-issues:

4.2 Stap 1 — Stationaire baseline (10 s — voor T2 en T3)

Wat je doet: chair staat helemaal stil, je laat 10 seconden data lopen. Markeer begin en eind met de knoppen in de Validatie-sectie van de app.

- ☐ Chair op MP1 (bottom-marker), motoren uit, chair geheel stil
- ☐ Wacht ~5 s zodat eventuele opstartfluctuaties verdwijnen
- ☐ Tik **Start baseline** ([BASELINE_START] regel verschijnt in log)
- ☐ Status verandert naar *BASELINE ACTIEF* (oranje)
- ☐ Wacht ~10 seconden (telefoon-timer, of gewoon op het oog)
- ☐ Tik **Eind baseline** ([BASELINE_END] regel)
- ☐ Status verandert naar *Niet actief (1 voltooid)*

De begin- en eind-timestamps komen automatisch in het log. Geen klok-tijden noteren.

Visuele observatie tijdens baseline (was er zichtbare beweging op het scherm?):

4.3 Stap 2 — Rijbeweging Bottom → Top

Wat je doet: de lift rijdt langzaam van bottom naar top. Bij elke marker tik je **MARK** in de app + maak je foto + noteer je in de tabel hieronder de getoonde positie.

- ☐ Joystick / startknop op de lift indrukken
- ☐ Lift begint te bewegen, vermoedelijk langzaam (firmware speed 20–50)

Bij elke marker (MP1 t/m MP5) doe je:

1. **Kijken** naar de fysieke chair en de markers
2. **Op het moment dat de chair fysiek over de marker rijdt:** tik de grote **MARK** knop in de Validatie-sectie. Op desktop kan ook *spatie*. Genereert [MARKER] n=N label="MPN" regel in log
3. Maak *foto* van het scherm: zorg dat de getoonde positie zichtbaar is
4. Lees de getoonde positie af (mm of % of normalized 0–1, wat de UI ook toont) en noteer in de tabel hieronder

Tip: tik *nét voor* de marker; menselijke reactietijd is ~200 ms en de chair is traag genoeg dat een tikje vroeg/laat niet veel uitmaakt. De counter in de app loopt automatisch: *Volgende: MP1* wordt *Volgende: MP2* na elke tik. De timestamp komt vanzelf in het log.

Marker	Foto?	Display toont	Observatie (smooth? glitch?)
MP1 (B→T)	☐ ja ☐ nee		
MP2 (B→T)	☐ ja ☐ nee		
MP3 (B→T)	☐ ja ☐ nee		
MP4 (B→T)	☐ ja ☐ nee		
MP5 (B→T)	☐ ja ☐ nee		

- ☐ Lift bereikt top station

Globale indruk B→T (smooth motion, geen jumps?):

4.4 Stap 3 — Rijbeweging Top → Bottom (terugweg)

Idem als stap 2, maar omgekeerd. Levert tweede meting per marker.

- ☐ Joystick activeren in tegengestelde richting

Marker	Foto?	Display toont	Observatie
MP5 (T→B)	☐ ja ☐ nee		
MP4 (T→B)	☐ ja ☐ nee		
MP3 (T→B)	☐ ja ☐ nee		
MP2 (T→B)	☐ ja ☐ nee		
MP1 (T→B)	☐ ja ☐ nee		

4.5 Stap 4 — Afsluiten

- ☐ Lift uit, sleutel uit
- ☐ App netjes stoppen (Ctrl+C in terminal, geen kill -9)
- ☐ Controleer logfile: open in editor, scroll door, verifieer aanwezigheid van [CSV], [MARKER], [BT_RTT] regels
- ☐ Tape van rail halen
- ☐ Foto's naar laptop overzetten (bij voorkeur direct, anders binnen 24 u)

Aantal [CSV] regels in logfile (ongeveer): _____

Aantal [BT_RTT] regels: _____

Aantal [MARKER] regels (verwacht: 10× + 2 baseline): _____

4.6 Observaties opschrijven (binnen 10 min na de run!)

Geheugen vervaagt snel. Schrijf nu op wat je zag, ook als het allemaal goed ging.

O1 — Visuele smoothness:

O2 — Anomalieën (BT drops, glitches, vreemd gedrag):

5 Post-run analyse (thuis / op kantoor)

5.1 T1 — Gemiddelde BT round-trip

Commando:

```
grep "\[BT_RTT\]" ~/logs/tablet/session_*.log | \
  awk -F"rtt_ms=" '{print $2}' | \
  awk '{sum+=$1; n++;} END {printf "T1 = %.1f ms (n=%d samples)\n", sum/n, n}'
```

T1 resultaat: _____ ms

n samples: _____

DV1(b) pass? ☐ ja (≤ 100 ms) ☐ nee

5.2 T2 + T3 — Encoder jitter, stationaire baseline

Extract de stationaire window uit het CSV-log. Eerst de hele CSV grep'en:

```
cat ~/logs/tablet/session_*.log | grep "\[CSV\]" | sed 's/\[CSV\] //' \
> ~/logs/csv_data.csv
```

Open `csv_data.csv` in een spreadsheet of in Python. Filter de regels waarvan de timestamp t tussen `[BASELINE_START]` en `[BASELINE_END]` timestamps in het log valt. Voorbeeld om die twee snel op te zoeken:

```
grep -E "\[BASELINE_(START|END)\]" ~/logs/tablet/session_*.log
```

Bereken binnen dat venster:

- $T2 = \max(raw_encoder) - \min(raw_encoder)$
- $T3 = \max(display_pos) - \min(display_pos)$, omgerekend naar pulses

T2 raw jitter (peak-to-peak): _____ pulses (dus $\pm$ _____ pulses)

T3 Kalman jitter (peak-to-peak): _____ pulse-equivalent

Ratio T2/T3: _____ $\times$

DV3(b) pass? ☐ ja (ratio ≥ 10) ☐ nee

5.3 T4 — Positie-error per marker

Stap 1 — bereken voor elke marker de *verwachte absolute positie*. Per marker:

1. Tel de lengtes van alle voorgaande rail-onderdelen op (straights: lees uit firmware / rail JSON; bochten: $13.09 \cdot n_{segs}$ mm).
2. Tel de gemeten offset (sectie 4) erbij.
3. Dat is de verwachte positie in mm vanaf bottom.

Stap 2 — haal de *getoonde positie* uit de log. Marker-passage regels:

```
grep "\[MARKER\]" ~/logs/tablet/session_*.log
```

`display_pos` in de log is normalized (0–1). Omrekenen naar mm: `display_pos  $\times$  rail_length_mm`.

Stap 3 — vergelijken en error berekenen.

Marker	Verwacht (mm)	Display B→T (mm)	Display T→B (mm)	Avg (mm)	Error (mm)
MP1					
MP2					
MP3					
MP4					
MP5					

T4 max error: _____ mm

T4 gemiddelde error: _____ mm

Rail-lengte was: _____ mm

threshold: ☐ 5 mm (≤ 5 m) ☐ 10 mm (langer)DV2(b) pass? ☐ ja (max $\leq$ threshold) ☐ nee

6 Verslag invullen

Open `project/main.tex`, ga naar Table tab:validation (rond regel 1100). Vervang [meet] per rij:

Verslag-rij	Bron	In te vullen	Pass/fail tekst
Rail length	sectie 4	_____ mm	—
Number of horizontal bends	sectie 4	_____	—
Position error at end station	MP1 of MP5	_____ mm	DV2(b): ☐ pass ☐ fail
Max position error at ref. points	T4 max	_____ mm	DV2(b): ☐ pass ☐ fail
Encoder jitter (raw, stationary)	T2	$\pm$ _____ pulses	—
Encoder jitter (after Kalman)	T3	_____ pulses	DV3(b): ☐ pass ☐ fail
Bluetooth round-trip time	T1	_____ ms	DV1(b): ☐ pass ☐ fail
Visual smoothness through bends	O1	korte zin	—

Logged events paragraaf (regel ~1126): vul werkelijke observaties O2 in.

Build PDF:

```
cd project && latexmk -pdf main.tex
```

Commit:

```
git add project/main.tex project/main.aux project/main.fdb_latexmk \
      project/main.fls project/main.log project/main.pdf
git commit -m "validate: fill measurement results from physical run"
```

7 Eerlijkheids-regels

- ☐ Bij failure (criterium niet gehaald): noteer de werkelijke waarde, markeer *fail*, en bespreek de oorzaak in de Conclusion van het verslag. Niet “een tweede run doen tot het past”.
- ☐ Eén run is genoeg voor proof-of-concept; N=1 is duidelijk in de Limitations vermeld.
- ☐ Bewaar de raw logfile in `logs/` (niet committen, te groot). Bewaar wel een geanonimiseerde samenvatting als `logs/run_summary.txt`.
- ☐ Niets verzinnen. Als T1 niet gemeten is wegens ontbrekende instrumentatie, rapporteer dat: “BT round-trip not measured in this run; instrumentation required for future validation.”

Afsluitende notities / lessons learned:

Versie van dit protocol: 28 mei 2026. Plak een ingevulde kopie bij het verslag op de map / sla op in `logs/`.